

Airbnb Pricing and Market Intelligence Platform

INF2006 Cloud Computing and Big Data Team Project

Class: EP2, Group G014

Deployment region: AWS Asia Pacific (Singapore), ap-southeast-1

Evidence dates: 1-4 October 2026

Deployment status: The stack described here is currently deployed in ap-southeast-1. It was rebuilt through Terraform on 4 October 2026 after the complete local preflight passed, loaded with the real listings dataset, and retested with a no-change Terraform plan. The encrypted remote Terraform backend remains available for reproducible redeployment.

Team: Lum Jia Jun (2500022), Nixon Lee Disheng (2500594), Madugula Adheesh (2500670), Leow Yi Hao Ignatius (2501538), and Wong Zhen Ho Brendan (2503427).

1. Problem, users and success criteria

Prospective and existing Airbnb hosts lack a simple way to evaluate a potential listing configuration before publishing it. They must reason about location, room type, property characteristics, capacity, host signals, and local price patterns across markets that use different currencies. Individual listing pages do not combine model-based scenario estimation, configuration comparison, historical market context, and private history in one focused workflow. A single global average is not useful because the supplied dataset covers ten cities and prices are recorded in each city's local currency.

The primary users are existing hosts reviewing a listing and potential hosts exploring whether a property configuration may be suitable for hosting. The guided workflow selects a city and neighbourhood, describes a potential listing, returns a model-backed nightly-price estimate, compares it with the city's historical market, and keeps up to three browser-local scenarios for configuration comparison. A signed-in user can save estimates and later retrieve only their own history. Visitors can explore descriptive city-level listing counts, median prices, and ratings, but the service deliberately remains a host decision-support tool rather than a booking platform.

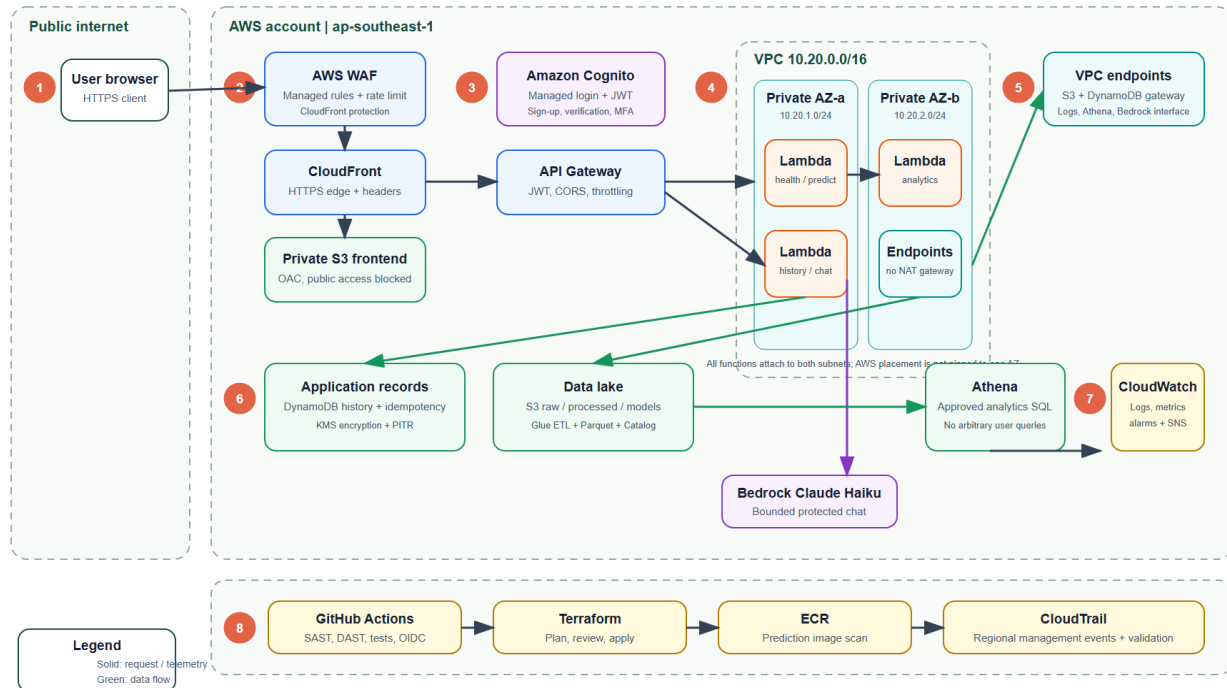
The project defines success as a working, reproducible, and evidence-backed workflow rather than maximum feature count. Functional success requires a CloudFront page, healthy API, real model inference, ten-city analytics, and authenticated history routes. Data success requires a profiled source, reproducible preprocessing and training, held-out metrics, and a cloud transform that produces queryable Parquet. Security success requires private S3 origins, HTTPS, least-privilege roles, Cognito JWT authorization, input validation, encrypted persistent stores, and evidence against named threats. Operational success requires structured logs, metrics, alarms, recovery configuration, and an honestly interpreted load test.

The service is deliberately scoped. Predictions cover the typical market up to a city-specific 99th-percentile boundary learned from training data. They are not property valuations, purchase recommendations, occupancy forecasts, profitability estimates, guarantees, or financial advice. The dataset does not contain acquisition prices, operating costs, regulation, tax, mortgage, booking-demand, or return data. Analytics values use different local currencies and cannot be compared as though all values were denominated alike. Success does not mean eliminating model error, handling unlimited traffic, or proving production readiness from one university development account.

2. Solution overview and architecture

INF2006 Airbnb Pricing and Market Intelligence Platform

Terraform-defined and previously verified architecture | ap-southeast-1 | Two-AZ Lambda VPC, WAF, CloudTrail, Cognito, and GitHub CI/CD



The solution uses managed and serverless AWS services. CloudFront is the only public frontend delivery layer and reads static HTML, CSS, JavaScript, and configuration objects from a private S3 bucket through Origin Access Control. Browser requests reach an API Gateway HTTP API. Public routes provide health, prediction, and market analytics. Cognito provides sign-up and sign-in through the hosted UI using the authorization-code flow with PKCE. API Gateway validates JWTs for saved predictions, history, and AI assistant access.

Five focused Lambda functions separate health, prediction, analytics, history, and AI responsibilities. The prediction function is a Lambda container image in ECR because the compatible NumPy, pandas, SciPy, scikit-learn, and model bundle exceed a practical small ZIP deployment. The history function queries DynamoDB by the verified Cognito subject. The analytics function runs one fixed Athena aggregate query, so clients cannot submit SQL. The separate chat function invokes the global Claude Haiku 4.5 inference profile only for JWT-authenticated users; it validates a 500-character prompt boundary, caps output at 220 tokens, and does not log prompt text.

The analytical data path uses a separate private S3 data lake. The uploaded raw listing CSV remains under `raw/listings/`. An AWS Glue 5.0 Spark job selects documented fields, rejects invalid required values, applies the same typical-market boundary used by modelling, and writes city-partitioned Parquet under `processed/listings/`. A Glue Catalog table uses partition projection for the ten observed cities. Athena runs in an enforced workgroup with encrypted output, CloudWatch metrics, a 1 GiB scan cutoff, and seven-day query-result expiry.

The architecture has two explicit trust areas: the public browser and the AWS account. HTTPS protects data in transit. IAM execution roles constrain each Lambda and the Glue job to required resources and operations. CloudWatch receives logs and metrics; API and function alarms target an encrypted SNS topic. DynamoDB point-in-time recovery provides a managed application-record recovery mechanism. Authenticated prediction writes require a caller-generated idempotency key, which is stored with a request fingerprint and response for safe retries. The current labelled diagram is stored at `evidence/architecture-current.png`.

3. Cloud service/deployment choices and trade-offs

The deployment model is public cloud. The service model combines FaaS for five Lambda functions, managed PaaS/serverless services for the API, identity, data, analytics, AI, edge, and operations layers, and a small IaaS networking responsibility for the VPC, private subnets, route tables, security groups, and endpoints. GitHub Actions is external SaaS delivery tooling. ECR stores the Lambda image but the platform does not run a CaaS service such as ECS, EKS, Fargate, or Kubernetes. Terraform was selected so resources, policies, limits, and outputs are versioned and reviewable. This boundary reduces server maintenance but creates AWS coupling and requires careful control of provider-managed behavior and quotas.

Lambda and API Gateway were selected instead of EC2 with an Auto Scaling Group and a reverse proxy. The workload is request-driven, low-volume, and uneven, so continuously running virtual machines would add patching, capacity planning, and idle cost. Lambda provides managed horizontal execution and per-request billing. Its trade-offs are cold starts, deployment package constraints, account concurrency limits, and a maximum request duration. The measured scientific Python cold start motivated 2 GB memory and a container image. The five functions now use private subnets across two availability zones with VPC endpoints for their required AWS services, while the approved account concurrency quota is 1,000.

S3 plus CloudFront was selected instead of a continuously running web server. The frontend is static, and CloudFront provides HTTPS delivery while Origin Access Control keeps the bucket private. The trade-off is cache invalidation after object updates. The deployment guide therefore includes an explicit invalidation command, and the smoke test verifies every asset's content type to catch cached fallback pages.

DynamoDB was selected instead of a relational database because the application has one dominant access pattern: retrieve recent predictions for an authenticated user in reverse time order. The partition key is `user_id`; the sort key combines an ISO timestamp and prediction ID. On-demand capacity avoids provisioned-capacity tuning for this small workload. The trade-off is that arbitrary relational queries are not supported and key design must follow access patterns.

S3, Glue, Parquet, the Glue Catalog, and Athena were selected instead of loading analytical records into the transactional table or operating a database cluster. Athena provides serverless SQL over S3 and Parquet reduces the scanned subset for the implemented query. Query cost still depends on bytes scanned, first-query latency is visible to users, and Glue has a minimum worker cost. A fixed query, scan cutoff, partition projection, and expiring outputs control these risks.

SageMaker, ECS, Kubernetes, RDS, and Redis were not selected. A VPC was added after review as a controlled network boundary for the Lambda functions, but a NAT Gateway was deliberately excluded because gateway and interface endpoints cover the required AWS services. Training remains reproducible offline while inference is integrated into the cloud workflow.

4. Implementation, data design and security controls

Terraform provisions the frontend, API, two-AZ Lambda VPC, private subnets, route tables, security groups, S3/DynamoDB gateway endpoints, Logs/Athena/Bedrock interface endpoints, Lambda functions, ECR repository, Cognito, DynamoDB, S3 data lake, Glue job and catalog, Athena workgroup, CloudWatch resources, SNS topic, and KMS key. Resource names share a project and environment prefix. S3 buckets enable Block Public Access, bucket-owner enforcement, encryption, and versioning. Frontend reads are restricted to the CloudFront distribution. Raw data is never committed to Git or managed as a Terraform object.

The framework-free frontend covers the potential-host workflow from listing input through estimate, historical market context, and three browser-local comparisons. A first-time-host preset does not assume prior ratings or hosting history. A bounded what-if panel reruns the model after changing amenities, guest capacity, or superhost status one input at a time. It is labelled as scenario analysis, not future-price forecasting. Differences appear only when currencies match. The browser retries one transient gateway or service-unavailable response while showing that the model is warming.

CloudFront adds a Content Security Policy, one-year HSTS, anti-framing, MIME-sniffing protection, and a strict referrer policy. The CSP limits scripts and styles to the application origin and browser connections to the regional API Gateway hostname pattern and exact Cognito domain. A global CloudFront WAF now adds AWS IP reputation rules, the AWS Common Rule Set, and a per-IP rate rule. A regional management CloudTrail records validated management events in a protected S3 bucket. A custom ACM certificate remains pending because the team has not supplied a domain and DNS validation ownership.

The prediction API accepts only the final model schema. It rejects missing or unknown fields, unsupported categories, invalid city-neighbourhood combinations, non-finite values, out-of-range coordinates, and unreasonable numeric values. It returns safe JSON errors without traces. Predictions are clipped to the supported market boundary and always include a disclaimer. Listing inputs are not written to logs.

Cognito uses email sign-in and verification, a 12-character mixed password policy, token revocation, and a public client without a secret. The browser generates a cryptographically random OAuth state and PKCE verifier, uses SHA-256 for the challenge, validates returned state, exchanges the code directly, and holds the ID token in session storage. This reduces code interception and long-lived browser persistence risk, although browser script compromise would still expose an active token.

API Gateway protects POST /predictions, GET /history, and POST /chat. Backend code derives the user ID only from the verified JWT sub; no request parameter can select another user. Saved records include the output, model version, timestamp, and safe listing and host signals needed for follow-up questions, but not coordinates. Chat projects only those fields, reads at most ten records, and reports the count used without exposing identifiers. Each Lambda retains a purpose-specific role.

The analytics endpoint does not accept query text or view parameters. It submits a constant aggregate query in a named Athena workgroup and converts typed results to a stable response. Descriptive outputs cover city listing volume, average and median price, and rating. Diagnostic outputs add distribution shape, capacity-price correlation, and observed superhost/non-superhost price difference; the interface labels these as cross-sectional associations rather than causal effects. The Lambda role is limited to that workgroup, catalog resources, processed objects, query-result objects, and its log group. The Glue role can read only the raw listing prefix and script, then read, write, or remove only processed listing objects.

CloudWatch alarms watch API 5xx responses, prediction, analytics, and chat Lambda errors, and health throttles. The dashboard charts chat duration and errors separately from core prediction traffic. Alarm actions publish to an SNS topic encrypted with a rotating customer-managed KMS key. The key policy permits account administration and limits CloudWatch publishing to this account's named alarms. The school email subscription is supplied as deployment configuration, confirmed by the recipient, and tested with a reversible alarm transition.

5. Analytics or AI/ML feature: data, method, evaluation and limitations

The dataset is the CC0 Airbnb Listings & Reviews release by mysarahmadbhat on Kaggle. Local files contain 279,712 listings and 5,373,143 review records across ten cities. Profiling records SHA-256 hashes, schemas, missingness, categories, price distribution, date coverage, and decoding issues. Reviews contain identifiers and dates but no text, so sentiment analysis was rejected. Prices are in each city's local currency.

The decision workflow covers four bounded analytics categories. Descriptive city summaries establish historical context. Diagnostic associations highlight market shape and relationships worth investigating without claiming causation. The predictive model estimates nightly price from city, neighbourhood, coordinates, property and room type, capacity, bedrooms, minimum nights, rating, host listing count, amenity count, instant bookability, superhost status, and identity verification. Identifier and free-text fields are excluded. Missing optional numeric fields are imputed. Categorical values are encoded inside the complete exported pipeline. The target uses $\log_{1p}$ because price is strongly right-skewed. Prescriptive support is deliberately narrow: the browser compares an estimate with its same-city median and suggests the next scenario to verify, but does not optimize occupancy, profit, or investment return.

A city-stratified 80/20 split protects representation across the ten markets. City-specific 99th-percentile price limits are learned from the training partition only, leaving 221,470 training rows and 55,363 held-out rows in V1 scope. A city median baseline, regularized linear model, and histogram gradient boosting model were compared. Selection uses the median per-city normalized MAE so one high-denomination currency does not dominate model choice.

Histogram gradient boosting performed best. On the held-out supported scope it achieved pooled MAE 191.836, RMSE 629.762, pooled local-currency R-squared 0.644, log-price R-squared 0.850, and median city-normalized MAE 0.592. The pooled currency metrics are reported for completeness but are less interpretable than per-city values. These are moderate results with material residual error, not evidence of high accuracy.

The submission also includes 500 deterministic synthetic listings and a single offline preflight command. It trains and evaluates all candidate pipelines without AWS credentials or the excluded 400 MB working dataset. Synthetic metrics are execution evidence only; the full-dataset metrics above remain the evaluation evidence.

The deployed Glue transform creates the analytical representation independently of training. A measured run processed the 158,497,169-byte listing CSV in 90 seconds and produced ten Parquet objects totalling 1,223,651 bytes. This difference reflects selected columns, filtering, and compression, so it is not presented as a pure format benchmark. A city summary query scanned 566,077 bytes, used 574 ms of Athena engine time, and completed in 697 ms. The API returned all ten summaries in about 1.3 seconds during the recorded check.

Limitations include historical staleness, missing values, absent demand and booking outcomes, mixed currencies, unmeasured market changes, and exclusion of extreme luxury listings. The model may encode historical geographic and host-related patterns. Users should compare outputs with current local evidence and should not use the estimate as the sole basis for financial or housing decisions.

6. Testing, scalability/resilience and monitoring results

The repository contains 41 passing Python unit tests covering health responses, prediction validation and response shape, authenticated persistence, idempotent retries, history isolation, analytics parsing, bounded and plain-text AI responses, mixed-currency history metadata, prompt-injection wrapping, safe failures, data profiling, training behavior, protected-route authorization, purpose-specific Lambda roles, restricted Lambda egress, and preservation of real WAF denial responses. JavaScript files pass Node syntax checks. Terraform formatting and validation pass. The offline preflight also validates manifest paths, trains all model candidates against deterministic synthetic data, and starts a loopback HTTP server to verify the real frontend plus local health, analytics, model-schema, and prediction routes before cloud deployment. The deployed smoke script verifies frontend assets and MIME types, health, real prediction, ten-city analytics, malformed-input rejection, and unauthenticated 401 responses for all three protected capabilities.

The chatbot now receives server-controlled platform facts plus a structured, newest-first summary of at most ten user-scoped DynamoDB records. Temperature-zero Haiku calls are still probabilistic service outputs, so a repeatable lexical evaluation checks rather than assumes behavior. All 16 local Bedrock scenarios and all 16 deployed Lambda scenarios passed: latest-record recall, stay-cost arithmetic, empty history, mixed currencies, city currencies, dataset and model limits, required MFA, investment boundaries, prompt injection, out-of-scope refusal, architecture, unsupported cities, unavailable analytics values, deployment-status uncertainty, and private-account boundaries. The deployed evaluator removed its synthetic rows, unauthenticated `/chat` returned 401, and recent CloudWatch events contained no tested prompt or history markers. These bounded results do not prove correctness for arbitrary future questions.

A JMeter 5.6.3 browser journey ran 25 visitors over 45 seconds against the three public pages and shared assets. All 929 requests passed at 20.3 requests per second, with 666.9 ms mean and 1,508 ms maximum response time. This bounded result verifies the tested CloudFront profile, not an unlimited scaling claim.

Security tests confirmed all four Block Public Access settings on both S3 use cases, anonymous object requests returned 403, and the data lake reported AES-256 default encryption. The Cognito authorization endpoint redirected to its hosted login page. ECR scan-on-push is enabled; prediction image tag `1.0.6` retains the updated base-image curl packages and its replacement scan completed with zero HIGH or CRITICAL findings. Unit tests confirm history queries use only the verified JWT subject.

IAM policy simulation added a service-to-service authorization check. Required prediction writes, history queries, and Glue raw-listing reads returned `allowed`. Prediction reads of raw S3 data, history table scans, and Glue reads of the unused raw-review prefix returned `implicitDeny`. This verifies selected blast-radius boundaries without treating same-account services as implicitly trusted.

The bounded live test sent 20 health requests at concurrency two. The recorded validation run returned 15 200 and five controlled 429 responses, with 231.26 ms median and 674.34 ms maximum latency. Earlier runs returned 18 200 and two 429, and 19 200 and one 429. The test passes only for 200 or 429 and fails for any backend error.

The earlier higher-concurrency finding is intentionally retained as historical evidence. After the account quota increased to 1,000, the controlled 100-request, concurrency-10 health test returned 53 200 and 47 429 responses, while the prediction test returned 62 200 and 38 429. No 5xx responses, Lambda errors, or Lambda-level throttles were observed. The results demonstrate controlled overload handling at the tested size, not unlimited capacity or DDoS protection.

DynamoDB continuous backups and point-in-time recovery both reported `ENABLED`. A destructive restore was not performed because DynamoDB restores into a new table and would require a tested cutover. Operational monitoring includes structured Lambda events without listing or prompt text, Athena scan and timing metrics, Glue job state, a dashboard, and five alarms. An alarm transition initially exposed a KMS policy failure. After a scoped customer-managed key was deployed, CloudWatch history reported successful execution of the encrypted SNS action, and the test alarm was reset to OK.

7. Cost, sustainability and operational considerations

The architecture minimizes always-on compute. Lambda, API Gateway, DynamoDB on-demand capacity, Athena, Glue jobs, S3, and CloudFront charge primarily by usage or stored volume. The largest intentional batch cost is the Glue Spark job, which runs only when the source data changes and is limited to two G.1X workers, a ten-minute timeout, and no retries. Training remains offline to avoid an unnecessary managed training service.

Cost controls are implemented rather than only documented. The Athena workgroup enforces a 1 GiB scan limit. Query results expire after seven days. ECR retains only three images. CloudWatch log groups retain 14 days. The CloudTrail bucket retains 90 days. The frontend uses low-cost static delivery. Data is partitioned and columnar. The AI route requires Cognito, uses bounded tokens, and has a tighter throttle than core routes. API throttling limits the development environment. Terraform supports teardown, although the data-lake bucket uses `force_destroy` to make intentional cleanup complete and must therefore be protected through normal change review. The original low-traffic estimate is retained for the pre-VPC baseline; the FinOps addendum in `evidence/cost-estimate.md` calls out the added two-AZ interface endpoint and WAF costs that must be measured in Cost Explorer.

Sustainability benefits come from avoiding idle servers and reducing repeated data scans. Parquet stores only analytical fields and the current query scans about 0.54 MiB. These choices reduce compute and storage activity for this workload, but no carbon measurement was performed and no universal sustainability claim is made. Glue startup overhead may outweigh benefits for very small one-off files; its value here is the reproducible cloud data-engineering path and integration with Athena.

Operational procedures include reviewing Terraform plans, using immutable ECR tags, invalidating CloudFront after frontend changes, running smoke tests, checking alarm state, and confirming the Glue run before expecting analytics. AWS credentials remain outside Git through the standard credential chain. Raw datasets, Terraform state, model binaries, environment files, and plan files are ignored.

For a longer-lived deployment, the team should monitor use of the approved 1,000 Lambda concurrency quota, retain ownership of the confirmed SNS recipient, preserve the remote encrypted Terraform backend and state locking, schedule backup-restore exercises, test cross-region recovery, and review current cloud spend. The university environment remains intentionally small and should be destroyed when evidence collection is complete.

8. Team contribution, ethical considerations and reflection

The EP2 Group G014 roster is Lum Jia Jun (2500022), Nixon Lee Disheng (2500594), Madugula Adheesh (2500670), Leow Yi Hao Ignatius (2501538), and Wong Zhen Ho Brendan (2503427). Nixon is recorded as Infrastructure and Cloud Deployment Lead. The remaining contribution, test-ownership, and reflection fields are left for each teammate to complete with genuine work evidence in `TEAM_CONTRIBUTIONS.md`.

The project uses OpenAI Codex for repository inspection, implementation support, Terraform, tests, documentation, and deployment verification. AI-assisted output was not accepted as evidence by itself. Claims were checked with unit tests, live HTTP requests, Terraform plans, AWS CLI output, CloudWatch metrics, and source review. The use and verification process is declared in `AI_USE_DECLARATION.md`.

The source dataset is listed as CC0 on Kaggle, but it represents historical platform data and may contain host or reviewer identifiers in raw files. Raw records are excluded from Git and remain in a private encrypted bucket. Identifier and free-text fields are excluded from the model and processed analytics. Reviewer IDs are not uploaded because the implemented analytics do not need `Reviews.csv`. Prediction history stores the Cognito subject and a limited set of listing attributes; coordinates are intentionally omitted.

Responsible interpretation is central to the design. Prices are historical, city currencies differ, availability is not treated as proven demand, and review sentiment is not fabricated. The estimator communicates uncertainty through its disclaimer and documented metrics. Geographic and host features may reproduce historical market differences, so the output should support, not replace, current local research and human judgement.

The strongest engineering lesson is that managed services do not remove the need to test boundaries. CloudFront caching initially served fallback HTML for asset paths until invalidation. A prediction deployment briefly returned 503 during transition. Load testing exposed an account quota, and alarm testing exposed an encryption-policy gap. Each issue was diagnosed with service state and logs, corrected where practical, and retained in evidence where limitations remain.

References

1. INF2006 Cloud Computing and Big Data, Team Project 1 brief, 2026. 2. mysarahmadbhat, "Airbnb Listings & Reviews," Kaggle, CC0 1.0 Public Domain. 3. AWS service configuration and behavior are evidenced by Terraform files and dated CLI results in evidence/. 4. scikit-learn documentation and BSD-3-Clause licensed implementation, <https://scikit-learn.org/>.